

Material Classification — CLI Reference Manual

A reference for operating the Material Classification command-line tool. The CLI classifies orthophoto rasters into **6 material classes** and writes a fused result raster plus its companion sidecar files.

This manual is for the operator who runs the CLI. It documents every flag, every configuration key, and the current output behavior, all cross-checked against the source (`cli.py` , `backend/app/pipeline.py` , `backend/app/core.py` , `backend/app/shapefile_config.py` , `backend/app/shapefile_resolver.py` , `backend/app/config.py`).

1. Overview

The Material Classification app reads georeferenced imagery (orthophotos) and labels each pixel as one of six materials. There are two ways to drive the same underlying pipeline:

- **Web app / GUI** (FastAPI + React) — interactive, point-and-click. Covered by `RUNNING_GUIDE.md` . *Not* the subject of this manual.
- **CLI** (`cli.py`) — scriptable, headless, batch-capable. **This manual.**

Quick start

```
cli.py C:\orthos\area_42.tif C:\results\area_42.tif
```

That single command runs the full 6-material MEA pipeline (it auto-enables `--mea --sam3-enabled`), pulls building/road geometry and the water mask from `shapefile_config.json` , and writes one fused result raster at the path you gave, with its `.xml` , `.txr` , and shared `all_imgs.txs` sidecars next to it.

You can also pass folders for recursive batch processing:

```
cli.py C:\orthos\ C:\results\
```

2. How it works (conceptually)

The 6 materials

Each MEA class is defined once in `backend/app/core.py` (`MEA_CLASSES`). They split into two groups by their `source` field:

Material	Color (hex)	Source	Painted from	Composite name
<code>BM_VEGETATION</code>	<code>#228B22</code>	<code>kmeans</code>	Color clustering (KMeans)	<code>GENVEGETATION</code>
<code>BM_SAND</code>	<code>#EDC9AF</code>	<code>kmeans</code>	Color clustering (KMeans)	<code>SAND</code>
<code>BM_SOIL</code>	<code>#654321</code>	<code>kmeans</code>	Color clustering (KMeans)	<code>SOIL</code>
<code>BM ASPHALT</code>	<code>#2D2D30</code>	<code>mask</code>	Roads (SDE shapefile → SAM3 → empty)	<code>ASPHALT</code>
<code>BM CONCRETE</code>	<code>#B4B4B4</code>	<code>mask</code>	Buildings (SDE shapefile → SAM3 → empty)	<code>CONCRETE</code>

Material	Color (hex)	Source	Painted from	Composite name
BM_WATER	#1C6BA0	mask	Water raster mask only	WATER

- **KMeans-source** (the three "natural" materials) are derived by clustering the raster's color/feature space. They have meaningful spectral signatures, so the KMeans pass assigns each cluster to one of these three.
- **Mask-source** materials have no useful spectral signature, so they are *not* clustered. Instead they are painted on top from external masks:
 - **Roads** (`BM ASPHALT`) and **buildings** (`BM CONCRETE`) come from the configured SDE geodatabase; if that yields nothing they fall back to SAM3 segmentation, and if that also yields nothing, the material is simply not painted.
 - **Water** (`BM WATER`) is painted only from a configured **water raster mask** (a georeferenced GeoTIFF, band 1 > 0 = water). There is no SAM3/vector fallback for water — if no `water_mask` is set, no water is painted.

The `classify_v6` flow

The MEA pipeline (`backend/app/pipeline.py:classify_v6`) runs in phases:

1. **KMeans classification** on the **3 kmeans-source materials only** (`BM VEGETATION` , `BM SAND` , `BM SOIL`). A strict 1:1 cluster→material assignment (Hungarian) prevents one material from absorbing several clusters. This produces an intermediate classified raster.
2. **Mask acquisition** for the three mask-source materials:
 - *Water*: the configured water-mask GeoTIFF (or the `--water-mask` override), reprojected/clipped to the ortho. Source label `raster` .
 - *Roads*: features for the raster footprint are pulled from the configured **SDE** roads layer; the LINE geometry is **buffered to road-width polygons** (see the 3-tier width logic in §6). If SDE resolves nothing, fall back to **SAM3**; if SAM3 is disabled or empty, no roads painted.
 - *Buildings*: same SDE → SAM3 → empty fallback chain.
3. **Mask fusion** paints the masks onto the KMeans output in a fixed bottom-up order: **water** → **roads** → **buildings**. Later masks win where they overlap, so **buildings beat roads, roads beat water**, and any mask beats the underlying KMeans color. (The legacy "soft-prior veto" is currently disabled — mask sources are treated as authoritative and always paint.)
4. **XML rewrite**: the KMeans pass wrote a `Composite_Material_Table` XML listing only the kmeans-source materials; after fusion the companion `.xml` is rewritten to list all **6** materials.

The fallback chains, stated plainly:

- **Roads / buildings**: SDE geodatabase → SAM3 → empty.
- **Water**: water raster mask only (no fallback). Paints only when a `water_mask` is configured (or `--water-mask` is passed).

3. Installation / environment

The CLI is a thin wrapper. The real work happens inside a bundled Python virtual environment that sits **next to** `cli.py` .

- **Always use the project venv's Python**: `.venv\Scripts\python.exe` . Invoke `cli.py` through it (or via `MaterialClassification_CLI.exe` , which finds the venv for you — see below).

- **Do not use bare `python`** . On the dev machine bare `python` resolves to a fresh Python 3.14 that is broken for this stack (missing `pkg_resources` , broken `triton-windows` , GroundingDINO failures). It will not run the backend.

The runnable install is wherever a folder contains **both** `cli.py` and `.venv` side by side. On this machine that is `C:\ClassificationApp\` (the dev checkout itself has no `.venv`).

The launcher (`MaterialClassification_CLI.exe`)

`cli_launcher.py` (frozen as `MaterialClassification_CLI.exe`) walks upward from the exe to find the project root (the directory containing `cli.py` + `.venv`), then runs `cli.py` inside that venv with your arguments passed straight through. It keeps the exe tiny (~5 MB) instead of bundling torch/transformers.

Before launching, it sets these environment variables (so HuggingFace models resolve from the local cache and the backend imports correctly):

Variable	Value
<code>HF_HUB_OFFLINE</code>	<code>1</code> (set if not already present)
<code>TRANSFORMERS_OFFLINE</code>	<code>1</code> (set if not already present)
<code>HF_HOME</code>	<code><project_root>\models\hf_cache</code>
<code>PYTHONPATH</code>	<code><project_root></code>

GPU is optional

Color clustering (KMeans + nearest-anchor) is accelerated on GPU when the CUDA runtime is present, and falls back to CPU automatically otherwise. There is nothing to toggle on the command line — the engine probes at runtime. See §8 for the acceleration chain and how to install the GPU pack.

4. Usage — two invocation styles

4.1 Simple positional form (recommended)

```
cli.py <input> <output>
```

- Auto-defaults to `--mea --sam3-enabled` (the same 6-material pipeline the GUI runs). When you use the positional form **without** `--classes` / `--mea` , the CLI prints a notice and enables MEA for you.
- Reads SDE (buildings/roads) config and `water_mask` from `shapefile_config.json` .
- `<input>` and `<output>` may each be a **file** or a **folder**. A folder input is processed **recursively** (all supported files in all subfolders).

Examples:

```
:: One ortho → one fused result raster
cli.py C:\orthos\area_42.tif C:\results\area_42.tif

:: Folder in / folder out (recursive batch)
cli.py C:\orthos\ C:\results\
```

If you pass positionals **and** `--classes`, the CLI stays in legacy (custom-count KMeans) mode and does **not** auto-enable MEA.

4.2 Flag-based form (full control)

```
cli.py --input <file_or_folder> --classes <N> [--output <path>]
      [--step <step>] [--mode <mode>] [--smoothing <value>]
      [--tile-size <px>] [--workers <N>] [--image-workers <N>]
      [--tiling] [--max-threads] [--no-spectral] [--no-texture] [--indices]
      [--detect-shadows | --no-detect-shadows]
      [--vector <path> ...] [--vector-class <name> ...]
      [--mea] [--no-sam3] [--water-mask <path>]
```

Pass INPUT/OUTPUT either positionally **or** via `--input` / `--output`, **not both** — mixing the two is rejected.

All flags

Flag	Default	Choices	Meaning
<code>--input</code> , <code>-i</code> PATH	—	—	Raster file (<code>.tif/.tiff/.jpg/.jpeg</code>) or folder. A folder is processed recursively. Use this <i>or</i> the positional INPUT, not both.
<code>--classes</code> , <code>-c</code> N	—	≥ 2	Number of material classes for legacy KMeans mode. Colors are deterministic by class index. Required for non-MEA runs; ignored when <code>--mea</code> or the simple positional form is used.
<code>--output</code> , <code>-o</code> PATH	next to input with a step-specific suffix	—	Output file or output folder. Use this <i>or</i> the positional OUTPUT, not both.
<code>--step</code>	<code>full</code>	<code>step1</code> , <code>step2</code> , <code>full</code>	<code>step1</code> = classification + export only; <code>step2</code> = vector rasterization only (expects an already-classified input); <code>full</code> = full pipeline.
<code>--mode</code>	<code>regular</code>	<code>regular</code> , <code>multispectral</code>	<code>regular</code> = RGB; <code>multispectral</code> = enables spectral indices automatically.
<code>--smoothing</code>	<code>none</code>	<code>none</code> , <code>median_1</code> , <code>median_2</code> , <code>median_3</code> , <code>median_5</code>	Post-classification median smoothing filter.
<code>--tile-size</code> PX	<code>512</code>	<code>256</code> , <code>512</code> , <code>1024</code> , <code>2048</code> , <code>4096</code> (or <code>-1</code> for default)	Tile side length in pixels. Non-standard values warn but are accepted.
<code>--workers</code> N	<code>-1</code> (auto = CPU core count)	—	Parallel tile workers within a single image. <code>-1</code> = automatic.

Flag	Default	Choices	Meaning
<code>--image-workers N</code>	<code>-1</code> (auto = <code>min(4, CPU count)</code>)	—	Parallel images in folder mode. <code>-1</code> = automatic.
<code>--tiling</code>	off	flag	Enable tile-based processing for large rasters (avoids OOM).
<code>--max-threads</code>	off	flag	Use all available CPU threads.
<code>--no-spectral</code>	off (spectral on)	flag	Disable spectral features.
<code>--no-texture</code>	off (texture on)	flag	Disable texture features.
<code>--indices</code>	off	flag	Enable spectral indices (e.g. NDVI). Auto-enabled in multispectral mode.
<code>--detect-shadows</code>	off	flag	Enable shadow detection, pre-processing balance, and class inference.
<code>--no-detect-shadows</code>	—	flag	Explicitly disable shadow detection (it is off by default anyway).
<code>--vector PATH</code>	<code>[]</code>	repeatable	A vector shapefile (<code>.shp</code>) to rasterize onto the result. Repeat for multiple layers.
<code>--vector-class CLASS_NAME</code>	<code>[]</code>	repeatable	MEA class name for the <i>n</i> -th <code>--vector</code> layer, in order (e.g. <code>BM_CONCRETE</code>). Only meaningful with <code>--mea</code> . Valid names: <code>BM_VEGETATION</code> , <code>BM_SAND</code> , <code>BM_SOIL</code> , <code>BM ASPHALT</code> , <code>BM_CONCRETE</code> , <code>BM_WATER</code> .
<code>--mea</code>	off	flag	Use the 6-material MEA preset and run the SAM3-first <code>classify_v6</code> pipeline (matches the GUI). Makes <code>--classes</code> optional/ignored.
<code>--no-sam3</code>	SAM3 on (when <code>--mea</code>)	flag	Disable the SAM3 mask stage in MEA mode (KMeans-only fallback for roads/buildings).
<code>--water-mask PATH</code>	—	—	Water-mask GeoTIFF (band 1 > 0 = water), painted directly as <code>BM_WATER</code> . Overrides the <code>water_mask</code> in <code>shapefile_config.json</code> for this run. <code>--mea</code> only.
<code>--examples , -e</code>	—	flag	Print the full built-in guide and exit.

`--classes` vs `--mea` : A run needs *either* `--classes N` (legacy custom-count KMeans) *or* `--mea` (the fixed 6-material schema). The simple positional form supplies `--mea` for you.

Worked examples

From the CLI's own `--examples` text:

```

:: Legacy KMeans, 3 classes
python cli.py --input photo.tif --classes 3

:: Folder, 5 classes, multispectral
python cli.py --input C:\images\ --classes 5 --mode multispectral

:: Classification-only (step1), custom tile size, output folder
python cli.py --input photo.tif --classes 4 --step step1 --tile-size 1024 --output C:\results\

:: Tiling on, 8 tile workers, median smoothing, texture off
python cli.py --input photo.tif --classes 3 --tiling --workers 8 --smoothing median_3 --no-tex

:: Folder batch with parallel tiles and parallel images
python cli.py --input C:\images\ --classes 5 --tiling --workers 4 --image-workers 3

:: Multispectral with shadow detection
python cli.py --input ms_image.tif --classes 5 --mode multispectral --detect-shadows

:: Simple positional form (auto --mea --sam3-enabled); SDE + water_mask from config
python cli.py C:\orthos\area_42.tif C:\results\area_42.tif

:: Same, folder in / folder out
python cli.py C:\orthos\ C:\results\

:: Flag-based MEA (same pipeline the GUI uses)
python cli.py --input photo.tif --mea --output C:\results\

:: MEA with an explicit water mask (overrides the config water_mask)
python cli.py photo.tif out.tif --water-mask water_mask.tif

:: MEA, KMeans-only (no SAM3, no SDE → roads/buildings/water empty)
python cli.py photo.tif out.tif --no-sam3

```

A couple more:

```

:: Large ortho: enable tiling and pick a tile size that fits RAM
cli.py C:\orthos\big.tif C:\results\big.tif --tiling --tile-size 1024

:: Recursive batch into a single output tree
cli.py C:\orthos\survey_2026\ C:\results\survey_2026\ --tiling --image-workers 2

```

5. Output files

A normal MEA run produces **exactly one** fused raster at the path you specify, plus its sidecars. The CLI consolidates so there is **no separate unfused or `_fused` file** left behind (the `single_fused_output` path is enabled for the CLI's MEA branch).

For a single-file run `cli.py in.tif C:\results\out.tif`:

File	What it is
<code>C:\results\out.tif</code>	The fused 6-material result raster (RGB; each material painted in its color).

File	What it is
<code>C:\results\out.xml</code>	<code>Composite_Material_Table</code> listing all 6 materials, ARGB colors.
<code>C:\results\out.txr</code>	The output's WGS-84 (EPSG:4326) bounding box sidecar.
<code>C:\results\all_imgs.txs</code>	A shared batch-load script written into the output folder.

Folder (batch) mode writes one output per input. The default suffix in MEA / full mode is `_full`, so `area_42.tif` → `area_42_full.tif` (plus its `.xml`, `.txr`, and the shared `all_imgs.txs` in the output folder). Step-specific suffixes when no explicit output file name is given:

<code>--step</code>	Suffix
<code>full</code>	<code>_full</code>
<code>step1</code>	<code>_classified</code>
<code>step2</code>	<code>_with_vectors</code>

If `--output` is a folder (or has no file extension), the CLI treats it as a directory and derives the output file name from the input stem + suffix, recreating the input's subfolder structure under it.

The XML format

After classification, `backend/app/core.py:_write_composite_material_xml` writes a `<stem>.xml` next to each output. It is a `Composite_Material_Table` with one `<Composite_Material index="N">` entry per material. The `<Name>` is the material's **composite name** (`BM_VEGETATION` → `GENVEGETATION`, `BM ASPHALT` → `ASPHALT`, etc.); the `<Color>` is ARGB — `#ff` plus the lower-case RRGGBB of the class color. Example entry:

```
<Composite_Material_Table>
  <Composite_Material index="1">
    <Name>GENVEGETATION</Name>
    <Color>#ff228b22</Color>
    <Primary_Substrate>
      <Thickness>1</Thickness>
      <Material>
        <Name>BM_VEGETATION</Name>
        <Weight>100</Weight>
      </Material>
    </Primary_Substrate>
  </Composite_Material>
  ...
</Composite_Material_Table>
```

6. Configuration reference

Two JSON files drive the CLI's behavior. Both live in the **config directory**: the project root in development, or **next to the executable** for the bundled exe (`backend/app/config.py:_config_dir()`).

6.1 `shapefile_config.json`

Defines where buildings/roads and water come from. Buildings and roads are pulled per-raster from an Esri enterprise geodatabase (the `sde` block); water is a single raster mask. Schema and defaults are in

backend/app/shapefile_config.py .

```
{
  "water_mask": "C:/data/water_mask.tif",
  "buildings": [],
  "roads": [],
  "water": [],
  "sde": {
    "enabled": true,
    "connection_file": "C:/data/conn.sde",
    "arcpy_python": "C:/Program Files/ArcGIS/Pro/bin/Python/envs/arcgispro-py3/python.exe",
    "tile_size_metres": 5000,
    "timeout_seconds": 1800,
    "road_width_attr": "WIDTH",
    "Road_Type_Attr": "RTYPE",
    "Road_Type_Key_MainRoad": "main",
    "Road_Type_Width_MainRoad_m": 7.5,
    "Road_Type_Key_SideRoad": "side",
    "Road_Type_Width_SideRoad_m": 4.0,
    "road_width_fallback_m": 2.0,
    "layers": {
      "buildings": "GDB.SCHEMA.BUILDINGS",
      "roads": "GDB.SCHEMA.ROADS"
    }
  }
}
```

Top-level keys

Key	Type	Default	Meaning
<code>water_mask</code>	string (path)	<code>""</code>	A georeferenced GeoTIFF where band 1 > 0 marks water . Reprojected/clipped to each ortho and painted as <code>BM_WATER</code> . <code>""</code> = no water painted.
<code>buildings</code> , <code>roads</code> , <code>water</code>	string arrays	<code>[]</code>	Legacy per-feature shapefile path lists. Unused in SDE mode; present for backward compatibility.

Use `--water-mask PATH` to override `water_mask` for a single run.

The `sde` block

Key	Type	Default	Meaning
<code>enabled</code>	bool	<code>false</code>	Turn SDE extraction on. When off (or when it resolves no features), roads/buildings fall back to SAM3.
<code>connection_file</code>	string (path)	<code>""</code>	Path to the <code>.sde</code> connection file.
<code>arcpy_python</code>	string (path)	<code>""</code>	Path to ArcGIS Pro's <code>python.exe</code> (the one with <code>arcpy</code>). The extractor runs as an <code>arcpy</code> subprocess.
<code>tile_size_metres</code>	number	<code>5000</code>	Splits the AOI bounds into tiles of this size (metres) to stay under the ~2 GB / ~1M-row shapefile ceiling.
<code>timeout_seconds</code>	number	<code>1800</code>	Ceiling (seconds) for the <code>arcpy</code> subprocess (default 30 min).

Key	Type	Default	Meaning
<code>road_width_attr</code>	string	<code>""</code>	Tier 1 road-width field — see below.
<code>Road_Type_Attr</code>	string	<code>""</code>	Tier 2 road-type/class field — see below.
<code>Road_Type_Key_MainRoad</code>	string	<code>""</code>	Value of <code>Road_Type_Attr</code> that marks a main road.
<code>Road_Type_Width_MainRoad_m</code>	number	<code>0.0</code>	Full width (metres) for main roads. ≤ 0 falls through.
<code>Road_Type_Key_SideRoad</code>	string	<code>""</code>	Value of <code>Road_Type_Attr</code> that marks a side road.
<code>Road_Type_Width_SideRoad_m</code>	number	<code>0.0</code>	Full width (metres) for side roads. ≤ 0 falls through.
<code>road_width_fallback_m</code>	number	<code>2.0</code>	Tier 3 fallback full width (metres).
<code>layers.buildings</code>	string	<code>""</code>	SDE layer path for buildings.
<code>layers.roads</code>	string	<code>""</code>	SDE layer path for roads.

Road width — the 3-tier fallback

SDE roads arrive as **LINE** geometry. A bare line would burn only a 1-pixel centreline, so `backend/app/shapefile_resolver.py` buffers each road line into a polygon. The **full width** of each road is resolved **per feature** in three tiers (`_road_width_m`), and the buffer applied to each side is **half** that width:

1. **Tier 1 — explicit width (`road_width_attr`).** If the feature has a value in the `road_width_attr` field that parses to a number > 0 , that value (in **metres**, full road width) is used directly. The field name must be ≤ 10 characters (shapefile limit). Missing/ `NaN` / ≤ 0 falls through to tier 2.
2. **Tier 2 — road-type width (`Road_Type_Attr`).** The feature's `Road_Type_Attr` value is matched (case-insensitive, whitespace-trimmed) against the configured keys:
 - equals `Road_Type_Key_MainRoad` → use `Road_Type_Width_MainRoad_m`;
 - equals `Road_Type_Key_SideRoad` → use `Road_Type_Width_SideRoad_m`.

A matched width is used only when it is > 0 ; otherwise it falls through. If both keys are configured equal, **main wins**. Blank keys or a blank feature value disable this tier for that feature.

3. **Tier 3 — fallback (`road_width_fallback_m`).** When neither tier 1 nor tier 2 resolves, the full width is `road_width_fallback_m` (default **2.0 m**).

The **buffer radius is full width $\div 2$** . For example, a 7.5 m full width buffers 3.75 m on each side of the centreline; the default 2.0 m fallback buffers 1.0 m each side. Widths are converted from metres into the raster CRS's units (so a 2 m road stays 2 m, not 2 degrees on a geographic CRS), and lines use flat end-caps to avoid overshooting endpoints.

With `Road_Type_Attr` and its keys left blank, the logic reduces to the original two-tier behaviour:
`road_width_attr` else `road_width_fallback_m`.

If the configured `road_width_attr` (or `Road_Type_Attr`) is not actually a column in the extracted features, the resolver logs a warning and that tier is skipped — it does not error out.

6.2 `app_config.json`

Backend/model settings (`backend/app/config.py`). Lives in the same config directory as `shapefile_config.json`.

```
{
  "sam3_local_dir": null,
  "hf_cache_dir": null,
  "offline_mode": false
}
```

Key	Type	Default	Meaning
<code>sam3_local_dir</code>	string null	<code>null</code>	Path to the local <code>sam3-main</code> folder (SAM3 weights/assets). <code>null</code> = use the default lookup.
<code>hf_cache_dir</code>	string null	<code>null</code>	Custom HuggingFace cache directory. When set, exported as <code>HF_HOME</code> / <code>HUGGINGFACE_HUB_CACHE</code> .
<code>offline_mode</code>	bool	<code>false</code>	Force fully offline operation. When <code>true</code> , sets <code>HF_HUB_OFFLINE=1</code> and <code>TRANSFORMERS_OFFLINE=1</code> so no network is attempted.

The CLI launcher already sets `HF_HUB_OFFLINE`, `TRANSFORMERS_OFFLINE`, and `HF_HOME` (see §3), so the CLI is offline-first regardless of this file.

7. Paths & topology

Location	Purpose
<code><root>\cli.py</code> , <code><root>\.venv\</code>	The CLI script and its Python venv. The root is the folder containing both.
<code><root>\.venv\Scripts\python.exe</code>	The interpreter to run everything with.
<code><root>\app_config.json</code>	Backend/model config (§6.2).
<code><root>\shapefile_config.json</code>	Buildings/roads/water config (§6.1).
<code><root>\models\</code>	Model assets root.
<code><root>\models\sam3\</code>	SAM3 weights/tokenizer assets (used by the SAM3 mask stage).
<code><root>\models\hf_cache\</code>	HuggingFace cache (<code>HF_HOME</code> points here). On offline stations, copy the <code>hub\</code> folder in here.
Output folder you pass	Where the fused <code>.tif</code> + <code>.xml</code> + <code>.txr</code> + <code>all_imgs.txs</code> land.

On this machine the runnable install root is `C:\ClassificationApp\` (the development checkout has the source and `models\sam3` but **no** `.venv`, so run the CLI from the install, not the dev tree).

8. Performance & tuning

The per-run knobs you control from the command line:

Knob	Effect
<code>--tile-size</code> <code>{256..4096}</code>	Bigger tiles = fewer tiles but more RAM per tile. Smaller tiles = lower peak memory.

Knob	Effect
<code>--tiling</code>	Turn tile-based processing on . The main lever for very large rasters — without it the whole raster is held in memory and can OOM.
<code>--workers N</code>	Parallel tile workers within one image (default = CPU core count).
<code>--image-workers N</code>	Parallel images in folder mode (default = <code>min(4, CPU count)</code>).
<code>--max-threads</code>	Use all CPU threads.

For a very large raster the main lever is **enable `--tiling` and choose a `--tile-size` that fits available RAM**. The engine can also auto-suggest a safe tile size based on current free memory (the web app uses `POST /suggest-tile-size` for this); on the CLI, start with `1024` and reduce if you see `OutOfMemory`.

GPU acceleration

Color clustering is GPU-accelerated when the CUDA runtime is available. To enable it on a target machine, install the GPU pack:

```
.venv\Scripts\python.exe -m pip install -r backend\requirements-gpu.txt
```

These wheels are pinned to **CUDA 12.4** (backward-compatible with newer drivers; required for the deployed A4000 / older-driver PCs). The acceleration chain, in priority order:

```
faiss-gpu > cupy > faiss-cpu > cuml > sklearn
```

A missing or broken GPU library is a **warning, not a failure** — the pipeline still runs on CPU (`sklearn`). There is no CLI flag to force GPU/CPU; it is probed at runtime and the chosen engine is printed in the run log.

Deliberately not changed (documented for awareness): the tile memory margin is intentionally conservative to avoid OOM on the deployed A4000, and there is no per-tile GPU dispatch. These are out of scope as performance tweaks.

9. Troubleshooting

Symptom	Likely cause / fix
<code>OutOfMemory</code> on a large image	Enable <code>--tiling</code> and pick a smaller <code>--tile-size</code> (e.g. 1024 → 512).
GPU not used (slow KMeans)	<code>backend\requirements-gpu.txt</code> not installed, or NVIDIA driver too old for the runtime wheels. CPU fallback (<code>sklearn</code>) is automatic; install the CUDA-12.4 GPU pack to accelerate.
SDE produces no roads/buildings	Check <code>sde.enabled</code> is <code>true</code> , and that <code>connection_file</code> , <code>arcpy_python</code> , and <code>layers.roads</code> / <code>layers.buildings</code> are correct and reachable. With SDE off/empty, roads/buildings fall back to SAM3 (and to nothing if <code>--no-sam3</code>).
Roads paint as thin lines / wrong width	Verify the road-width config (§6.1): is <code>road_width_attr</code> a real column? Are <code>Road_Type_Attr</code> + keys + main/side widths set? Otherwise the 2 m fallback applies.

Symptom	Likely cause / fix
Water not painted	<code>water_mask</code> is unset, the path is wrong, or the GeoTIFF's band 1 is not <code>> 0</code> over water. Use <code>--water-mask PATH</code> to test a specific mask. Water has no SAM3 fallback.
<code>ERROR 1: PROJ: proj_identify: Cannot find proj.db</code>	The venv's <code>pyproj</code> data is missing/corrupt — reinstall <code>pyproj</code> into <code>.venv</code> .
<code>ModuleNotFoundError</code> / import errors	You ran bare <code>python</code> instead of <code>.venv\Scripts\python.exe</code> . Use the venv interpreter (or <code>MaterialClassification_CLI.exe</code>).
SAM3 / AI extraction fails	Model weights missing from <code>models\hf_cache\hub\</code> (offline) or a <code>triton</code> import issue. Confirm the HF cache is populated; on Windows <code>triton</code> is mocked so OWLv2+SAM2 remain available as a fallback.
Run leaves two rasters (<code>out.tif</code> + <code>out_fused.tif</code>)	Should not happen on the current CLI — the MEA branch consolidates to a single fused output. If you see it, you are likely on an old build or driving the backend directly rather than through <code>cli.py</code> .

This manual is generated from `docs/CLI_GUIDE.md`. For the interactive app and build/deploy topology, see `RUNNING_GUIDE.md` and the `docs/` reference set (`ARCHITECTURE.md`, `GPU_ACCELERATION.md`, `TILING.md`, `API_REFERENCE.md`).